

Tarea - TIA-05 (Unidad 3 - Tarea 1)
Implementación de un sistema de consultas y vistas

- Tarea en Equipo
- Peso: 20%
- Práctica. Caso de Estudio
- DML - Manipulación de Datos
- Proyecto de Aula

MIEMBROS DEL EQUIPO:

- Líder
- Miembro: Laura Carolina Cadena Martinez

Descripción de la Tarea

1. Contextualización

En el desarrollo del proyecto del sistema de Red Social Pascualina, se ha identificado que uno de los principales riesgos en la construcción de sistemas de información no radica únicamente en la implementación técnica, sino en errores arrastrados desde las etapas iniciales del modelado conceptual y lógico. En este mismo orden de ideas, esos errores se pueden evidenciar en el modelo físico y afectar la manipulación de los datos y la construcción de consultas:

- Una base de datos física mal relacionada (referencias entre tablas)
- Sin controles para evitar inconsistencias y embasuramiento de datos

Estos problemas impactan directamente la calidad de la información, generando bases de datos:

- Vulnerables
- Inconsistentes y difíciles de mantener
- Ineficientes y de baja calidad
- Propensas a errores en consultas

Por esta razón, antes de iniciar la implementación física, es obligatorio realizar un proceso de: Revisión, corrección y mejora del modelo físico. Este proceso debe garantizar que la base de datos física sea:

- Coherente y correcta estructuralmente
- Protegida para evitar inconsistencias durante procesos de actualización
- Amigable y flexible para realizar consultas con información de calidad

En este contexto, el reto consiste en tomar el modelo físico previamente construido para el sistema de Red Social Pascualina y transformarlo en un modelo completamente funcional, implementado en un SGBD, para poder realizar operaciones de inserción, modificación, eliminación y consultas a través del lenguaje de manipulación de datos (DML).

Es importante que al final se realicen pruebas para verificar el cumplimiento de las propiedades ACID (Véase Anexo A)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

2. Propósito de la Tarea

El objetivo de esta tarea es que los estudiantes:

- Detecten errores en su propio modelo físico y corregirlos. Justifiquen mejoras
- Realicen correctamente operaciones DML: INSERT, DELETE, UPDATE y SELECT
- Implementen operaciones DML en un SGBD (PostgreSQL en este caso)
- Implementen de vistas (VIEWS)
- Utilicen el relacionamiento entre tablas (JOIN), Funciones de agregación y consultas con parámetros.
- Verifiquen las propiedades ACID para establecer la calidad del modelo propuesto

3. Instrucciones de realización de la tarea

Experimentar en el SGBD establecido (PostgreSQL), de acuerdo con las necesidades de la red Social Pascualina. Luego, utilizando una herramienta gráfica (pgAdmin4), se enfocarán en la correcta manipulación del modelo, prestando especial atención al correcto funcionamiento de las instrucciones de inserción, actualización, eliminación y consulta. El proceso incluye una etapa de presentación y justificación de las decisiones de modelado, que permitirán la retroalimentación grupal guiada por el docente, para refinar el trabajo antes de la entrega final. Para evidenciar su aprendizaje, deberán organizar su trabajo en un repositorio de Git que contendrá varios entregables clave:

- Un informe técnico que justifique sus decisiones. Nota: Debe incluir las mejoras realizadas al Modelo Físico para permitir las consultas realizadas. Inventario de tablas definitivo
- El conjunto de scripts de manipulación solicitados en los formatos de plantilla suministrados.
- Análisis de los resultados, conclusiones generales y reflexiones individuales
- Un video corto donde sustenten el trabajo realizado y las conclusiones individuales que reflejen su aprendizaje personal. **DEBE MOSTRARSE EL ESTUDIANTE E IDENTIFICARSE CON SU NOMBRE. DEBE MOSTRAR CÓDIGO EN EJECUCIÓN -> Mostrar ejecución de scripts de creación, “poblamiento” y manipulación en el SGBD.**

Esta tarea les permitirá desarrollar competencias esenciales como el reconocimiento de sentencias DML y la implementación operaciones de manipulación de una base de datos, habilidades fundamentales para cualquier desarrollador de software. Adicionalmente, tendrá la oportunidad de realizar operaciones de auditoría a la base de datos para verificar su validez y calidad.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

4. Criterios de entrega y valoración de la tarea

Los entregables finales para esta tarea, serán almacenados bajo la siguiente arquitectura: **Carpeta raíz:**

Tarea 5. En esta carpeta estarán dos carpetas con los entregables: Informe, Resultados, Scripts y Video.

Carpetas	Contenido requerido	Observaciones
/tarea5	Todas las subcarpetas y archivos de la tarea.	No aplica
tarea5/Informe	Informe detallado (plantilla suministrada)	.pdf
tarea5/DDL	Diccionario de Datos (MEJORADO) Script de creación de las tablas de la base de datos	.xlsx .sql
tarea5/DML	Script de poblamiento de la base de datos Script de respaldo de la base de datos Todos los Scripts DML de manipulación de la base de datos Script de la VIEW Scripts de verificación de propiedades ACID	.sql
tarea5/video	Video corto (5-10 minutos) con todos los miembros explicando una parte del trabajo y las conclusiones individuales.	Enlace a YouTube

Estos son las plantillas de formato a utilizar OBLIGATORIAMENTE. Solamente debe cambiar la “X” por el Equipo de estudiantes

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Evidencias de aprendizaje evaluadas

Evidencia	Habilidad evaluada
Informe con el paso a paso de la propuesta de solución, teniendo en cuenta: ● Corrección y presentación de las tablas definitivas de la Base de Datos ● Creación de la base de datos definitiva mejorada (CREATE) ● Poblamiento de las principales tablas indicadas (INSERT) ● Operaciones de respaldo de las tablas de una base de datos ● Operaciones de actualización de la base de datos (UPDATE) ● Operaciones de eliminación de información de la base de datos (DELETE) ● Operaciones de consulta de la información de la base de datos (SELECT) ● Operaciones de consulta con vistas (VIEWS) ● Operaciones para validar propiedades ACID. ● El informe debe referenciar los scripts DML y explicar su estructura. ● Conclusiones individuales sobre el proceso de solución, destacando dificultades, dudas, aspectos relevantes del proceso.	Capacidad de análisis y redacción técnica Reflexión crítica y aprendizaje personal.
Tipos de dato SQL y NoSQL para big data e IoT con justificación técnica del uso de tipos de datos adecuados para escenarios de big data e IoT Ejemplo: campo JSON para las coordenadas geográficas u optimización del tamaño del dato para captar señales de sensores en IoT.	Comprensión y utilización de tipos de datos que representen una relación con soluciones big data e IoT.
Modelo Analítico implementado a través de operaciones DML: ● Inventario completo de tablas de la base de datos. ● Tablas con nombres según las reglas y la nomenclatura. ● Campos correctamente identificados con tamaño y tipo asociado. ● Relaciones adecuadas al caso (claves primarias y foráneas). ● Uso correcto las reglas, nomenclatura y estándares para BD físicas.	Comprensión del Lenguaje de Manipulación de Datos (DML)
Participación en Foro de discusión y seguimiento de tareas	Comunicación asíncrona y trabajo en equipo
Video de sustentación (presentarse con imagen y nombre. Mostrar y explicar código en ejecución)	Comunicación oral y trabajo en equipo
Específicos	Competencia en gestión de versiones y organización digital.

ATENCIÓN: Analicen los criterios de evaluación y verifiquen su cumplimiento así como la ponderación de los ítems en la RÚBRICA.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Inventario de Tablas OBLIGATORIAS

“Sin estas tablas no podrá responder a las consultas y operaciones DML”

- **Usuario.** Se refiere a los usuarios del sistema de información de la Red Pascualina. Un usuario debe tener asociado un tipo de usuario. *NOTA IMPORTANTE:* además de tener un tipo de usuario, debe tener un rol dentro de la Red.
- **Rol.** Los roles pueden ser los siguientes: **administrador** (administra la configuración de la Red en el BackOffice), **auxiliar** que apoya ciertas operaciones en el BacOffice, **miembro** (ingresa a la Red y participa en todas las actividades inherentes a este) y **visitante** (usuarios especiales bajo evaluación que frecuentan la Red Social. Los ingresa el usuario Administrador. **Véase Anexo A**
- **Tipo de Usuario.** Estos son potenciales tipos de usuario: estudiante, docente, empleado, empresario, invitado, público en general, entre otros. Sirve para clasificación y estadística. Los ingresa el Usuario con Rol de Administrador. **Véase Anexo A**
- **Perfil:** compartir su área de estudio, intereses, habilidades en diferentes disciplinas o deportes, etc. Estos perfiles permiten **conectar con compañeros para** encontrar y seguir a otros estudiantes con intereses y/o actividades similares. Contiene un campo tipo JSON o JSONB. No se utilizará en esta tarea pero se debe crear.
- **Publicación:** compartir preguntas sobre tareas, recursos útiles, noticias de la industria tecnológica o incluso memes relevantes para la vida universitaria, entre otros. Las ingresa el usuario participante.
- **Comentario:** comentarios que se hacen a las publicaciones. Ejemplo: un estudiante publica una nota sobre la participación de Colombia en el Mundial de Fútbol; otros usuarios pueden comentar esta publicación. **NOTA: OJO!** diferenciar entre el usuario que publica y el usuario que comenta.
- **Grupo:** los grupos pueden ser de estudio para materias específicas, equipos para hackatones o clubes de interés (videojuegos, programación competitiva, etc.). También los empleados pueden tener sus grupos así como los docentes y los empresarios. Un grupo debe tener asociado un “Perfil”. Los ingresa el usuario participante.
- **Tipo de Servicio:** cambalache, cursos, tutorías, mantenimiento de vehículos, eventos, entre otros. Los tipos de servicio los registra el usuario con rol de administrador o auxiliar (si se le concede el permiso para esto). *NOTA:* Es general para todos. Sirve para clasificación y estadística. Los ingresa el Usuario con Rol de Administrador. **Véase Anexo B**
- **Servicio:** Los servicios los ofrece un usuario. Debe tener asociado un “Tipo de Servicio” con su descripción y precio (duración y ubicación si aplica). Los ingresa el usuario participante.
- **Tipos de Producto:** libros, motocicleta, ropa, ticket para concierto, entre otros. Sirve para clasificación y estadística. Sirve para clasificación y estadística. Los ingresa el Usuario con Rol de Administrador. **Véase Anexo C**
- **Producto:** los usuarios pueden postear productos para la venta y otros usuarios pueden comprar. El producto debe tener asociado un “Tipo de Producto”. Sigue el mismo formato de los servicios y tipos de servicio. Los ingresa el usuario participante.
- **Tipos de Evento:** al igual que los anteriores casos de servicio y productos, se cumple lo mismo. El usuario Administrador ingresa los tipos de evento. Sirve para clasificación y estadística. Los ingresa el Usuario con Rol de Administrador. **Véase Anexo D**
- **Evento.** Los eventos pueden ser reuniones de estudio, talleres, actividades sociales, concursos de empleo, conferencias de los docentes, entre otros. El evento debe tener asociado un tipo de evento. El usuario que publica el evento es diferente a los usuarios que participan en el mismo. Los ingresa el usuario participante.

Es importante que se entienda que estas tablas implican relaciones. Por lo tanto, debe incluir las tablas que permiten que esta Red Social funcione. Por ejemplo, tenemos las siguientes tablas de relación necesarias:

- Un usuario publica un evento (**tabla evento**). Debe asociar el tipo de evento (**tabla evento_tipo**) y los usuarios se suscriben y participan de ese evento (**tabla evento_usuarios**). Esta última tabla debe tener el id_evento, id_usuario y cualquier otro campo necesario que requiera el evento. Es decir, después de que concluya el evento puede darle un “like” o hacer un comentario sobre el mismo. ¿Es importante incluir la fecha y hora de ese comentario? ¿Que otros campos cree Ud. que serían importantes?

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

- El ejemplo anterior es una muestra de la misma situación para “Servicio” y “Producto”.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Informe con resultados

Equipo "X"

(TIA-5: Unidad 3 - Tarea 1)

1.- Diccionario de Datos definitivo (Mejorado)

Autoguardado

20261-bd1-g100-tia-05-equipo-10-inv-diccionario... • Guardado en Este PC ▾

Buscar

Archivo

Inicio

Insertar

Dibujar

Disposición de página

Fórmulas

Datos

Revisar

Vista

Automatizar

Complementos

Ayuda

Acrob

Pegar

Portapapeles

Fuente

Alineación

Número

Estilos

Formato condicional ▾

Dar formato como tabla ▾

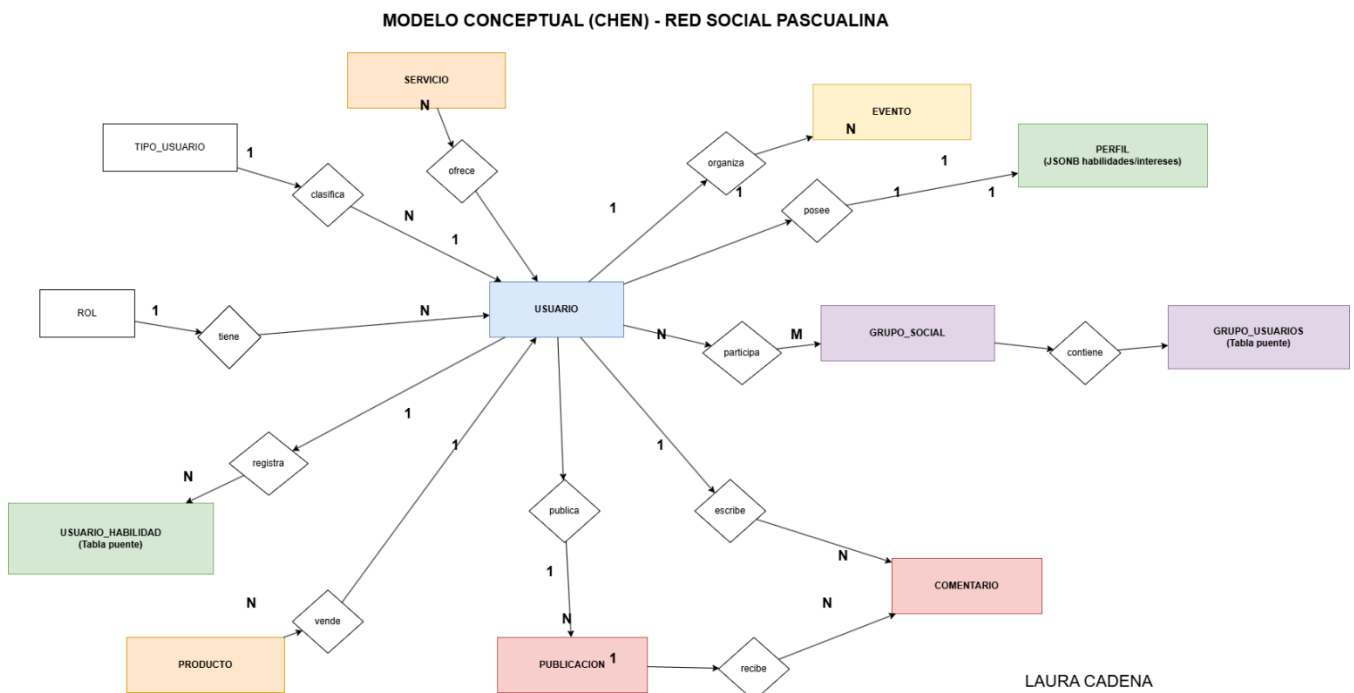
Estilos de celda ▾

Y144

<

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

INVENTARIO DE TABLAS				
#	Tabla	Descripción Tabla	Observaciones	
1	rol	Almacena los roles de los usuarios	Tabla independiente	
2	tipo_usuario	Clasifica los tipos de usuarios	Tabla independiente	
3	usuario	Guarda la información principal de los usuarios	Relacionada con rol y tipo_usuario	
4	perfil	Contiene información extendida del usuario	Usa JSONB	
5	publicacion	Registra publicaciones de usuarios	Depende de usuario	
6	comentario	Guarda comentarios de publicaciones	Relación usuario-publicación	
7	grupo_social	Almacena grupos académicos o sociales	Depende de usuario	
8	evento	Registra eventos estudiantiles	Depende de usuario	
9	servicio	Registra servicios publicados	Depende de usuario	
10	producto	Registra productos publicados	Depende de usuario	
11	grupo_usuarios	Relaciona usuarios con grupos	Tabla puente	
12	usuario_habilidad	Relaciona usuarios con habilidades	Tabla puente	
13				



2.- Script de “*creación*” de las tablas definitivas Bases de Datos (CREATE)

Script de creación de las tablas definitivas Bases de Datos (CREATE)

Introducción

Para el desarrollo de la base de datos de la red social estudiantil pascualina se implementó un modelo físico mejorado utilizando PostgreSQL como Sistema Gestor de Bases de Datos. El objetivo principal fue construir una estructura organizada, escalable y normalizada que permitiera almacenar la información de usuarios, publicaciones, comentarios, grupos sociales, eventos, productos y servicios dentro de la plataforma.

Durante el diseño del modelo físico se definieron tablas independientes, tablas dependientes y tablas puente para resolver relaciones muchos a muchos. Asimismo, se implementaron claves primarias (PK), claves foráneas (FK), restricciones de unicidad (UK) y tipos de datos apropiados según la naturaleza de la información almacenada.

El script de creación fue organizado respetando el orden de dependencias entre tablas para evitar errores de integridad referencial durante la ejecución.

Explicación técnica de la estructura creada

La estructura física de la base de datos fue diseñada siguiendo principios de normalización y buenas prácticas de modelado relacional.

Las tablas independientes creadas inicialmente fueron:

- *rol*
- *tipo_usuario*
- *grupo_social*

Estas tablas no dependen de otras entidades y permiten almacenar información base del sistema.

Posteriormente se crearon las tablas dependientes:

- *usuario*
- *perfil*
- *publicacion*
- *comentario*
- *servicio*
- *producto*
- *evento*

Estas entidades contienen claves foráneas relacionadas con la tabla usuario y otras relaciones del sistema.

Finalmente se implementaron tablas puente para resolver relaciones de muchos a muchos:

- *grupo_usuarios*
- *usuario_habilidad*

Estas tablas permiten representar múltiples asociaciones entre usuarios y otras entidades del sistema.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Justificación de tipos de datos

Se utilizaron diferentes tipos de datos según la necesidad de almacenamiento:

- *INT: utilizado para identificadores numéricos y claves primarias.*
- *VARCHAR: utilizado para nombres, correos y textos cortos variables.*
- *TEXT: utilizado para contenidos extensos como publicaciones y comentarios.*
- *DATE: utilizado para fechas específicas.*
- *TIMESTAMP: utilizado para registrar fecha y hora exacta de publicaciones.*
- *JSONB: implementado en PostgreSQL para almacenar intereses y habilidades del perfil de usuario de manera flexible y optimizada.*

El uso de JSONB permite manejar estructuras semiestructuradas relacionadas con habilidades e intereses estudiantiles, simulando escenarios de Big Data y analítica flexible.

Manejo de claves primarias y foráneas

Cada tabla posee una clave primaria definida mediante PRIMARY KEY para garantizar unicidad de registros.

Ejemplo:

- *id_usuario INT PRIMARY KEY*

Las relaciones entre tablas fueron implementadas mediante claves foráneas para mantener la integridad referencial.

Ejemplo:

- *FOREIGN KEY (id_usuario)*
- *REFERENCES usuario(id_usuario)*

Esto asegura que no existan registros huérfanos dentro de la base de datos.

Restricciones implementadas

Se implementaron restricciones adicionales para garantizar consistencia en los datos:

- *UNIQUE para evitar correos duplicados.*
- *NOT NULL en campos obligatorios.*
- *Restricciones FK para validar relaciones.*
- *Uso de tablas puente para relaciones N:M.*

Orden de creación de tablas

Las tablas fueron creadas respetando el siguiente orden:

- 1. Tablas independientes*
- 2. Tablas principales*
- 3. Tablas dependientes*
- 4. Tablas puente*

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

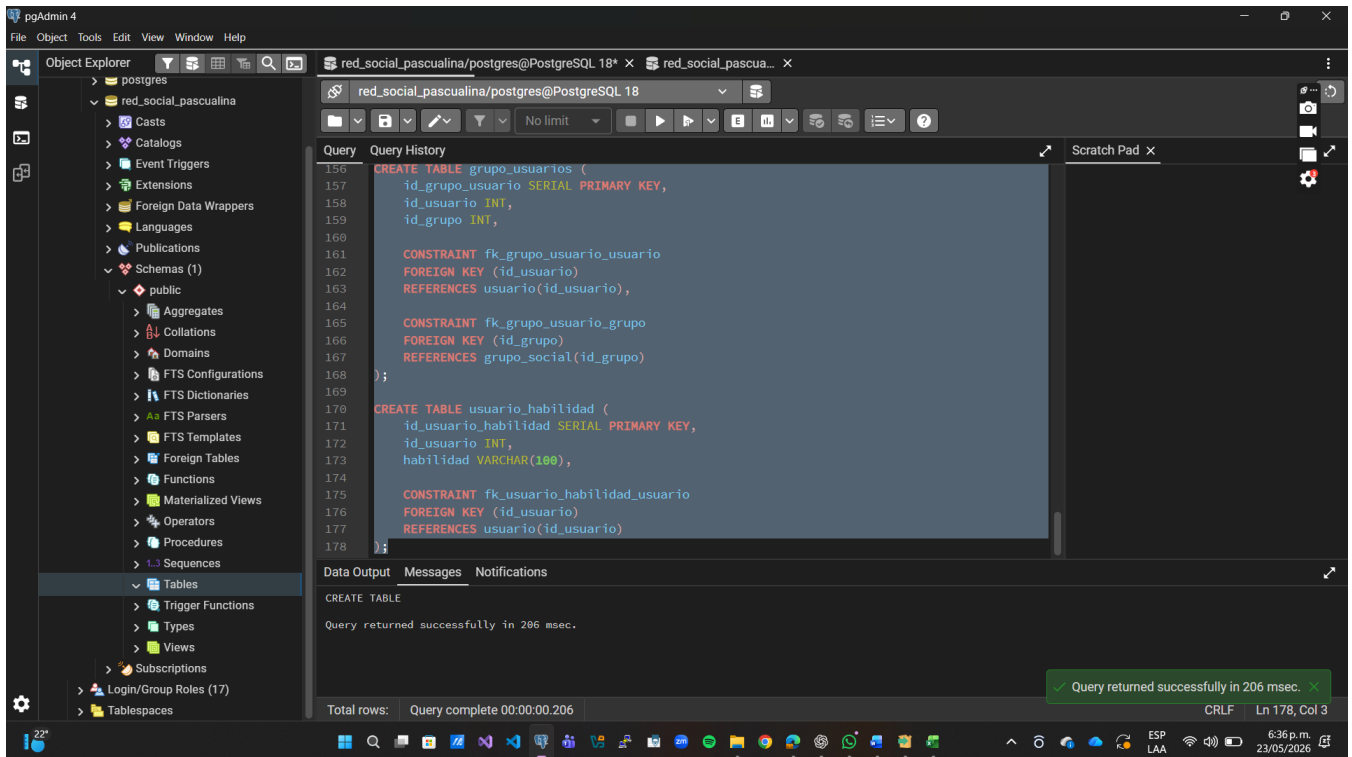
Este orden permitió ejecutar correctamente el script sin errores de dependencia.

Resultado de ejecución

El script SQL fue ejecutado exitosamente en PostgreSQL mediante pgAdmin4, verificando:

- *Creación correcta de tablas*
- *Relaciones entre entidades*
- *Integridad referencial*
- *Compatibilidad de tipos de datos*
- *Implementación de restricciones*

La ejecución final del modelo físico permitió obtener una base de datos funcional y preparada para operaciones DML y consultas analíticas posteriores.



The screenshot displays the pgAdmin 4 application window. On the left, the 'Object Explorer' pane shows the database structure, including the 'public' schema and its tables. The main pane shows a SQL script being executed. The script contains two CREATE TABLE statements with foreign key constraints. The first table is 'grupo_usuario' and the second is 'usuario_habilidad'. The 'Data Output' pane at the bottom shows the message 'Query returned successfully in 206 msec.' and 'Query complete 00:00:00.206'.

```
156 CREATE TABLE grupo_usuario (
157     id_grupo_usuario SERIAL PRIMARY KEY,
158     id_usuario INT,
159     id_grupo INT,
160
161     CONSTRAINT fk_grupo_usuario_usuario
162     FOREIGN KEY (id_usuario)
163     REFERENCES usuario(id_usuario),
164
165     CONSTRAINT fk_grupo_usuario_grupo
166     FOREIGN KEY (id_grupo)
167     REFERENCES grupo_social(id_grupo)
168 );
169
170 CREATE TABLE usuario_habilidad (
171     id_usuario_habilidad SERIAL PRIMARY KEY,
172     id_usuario INT,
173     habilidad VARCHAR(100),
174
175     CONSTRAINT fk_usuario_habilidad_usuario
176     FOREIGN KEY (id_usuario)
177     REFERENCES usuario(id_usuario)
178 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 206 msec.

Total rows: Query complete 00:00:00.206

CRLF Ln 178, Col 3

6:36 p.m. 23/05/2026

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3.- Script de “**poblamiento**” de la Bases de Datos (**INSERT**)

Script de “poblamiento” de la Base de Datos (**INSERT**)

Introducción

El poblamiento de la base de datos se realizó mediante sentencias INSERT INTO con el objetivo de simular el funcionamiento real de una red social estudiantil universitaria.

Se registraron datos relacionados con usuarios, roles, tipos de usuario, perfiles académicos, publicaciones, comentarios, grupos sociales, productos, servicios, eventos y habilidades estudiantiles.

El propósito del poblamiento es permitir la ejecución posterior de consultas DML, generación de vistas y análisis de información dentro del sistema.

Los registros insertados respetan las relaciones establecidas mediante claves foráneas y garantizan consistencia e integridad referencial en la base de datos.

```
1 TRUNCATE TABLE
2 usuario_habilidad,
3 grupo_usuarios,
4 comentario,
5 publicacion,
6 evento,
7 producto,
8 servicio,
9 perfil,
10 usuario,
11 tipo_usuario,
12 rol
13 RESTART IDENTITY CASCADE;
14
15 -- =====
16 -- TABLA rol
17 -- =====
18
19 INSERT INTO rol (nombre)
20 VALUES
21 ('Administrador'),
22 ('Moderador'),
23 ('Estudiante');
```

Tabla	Descripción	Registros
rol	Registro de roles del sistema	4
tipo_usuario	Clasificación de tipos de usuarios	9
usuario	Registro principal de usuarios	5
perfil	Información académica y habilidades en JSONB	5
publicacion	Publicaciones realizadas por usuarios	3
comentario	Comentarios realizados en publicaciones	3
grupo_social	Registro de grupos académicos y sociales	3
grupo_usuarios	Relación entre usuarios y grupos	4
servicio	Servicios ofrecidos por usuarios	3

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

<i>Tabla</i>	<i>Descripción</i>	<i>Registros</i>
<i>producto</i>	<i>Productos publicados por usuarios</i>	3
<i>evento</i>	<i>Eventos creados por usuarios</i>	3
<i>usuario_habilidad</i>	<i>Registro de habilidades de usuarios</i>	4

Explicación del poblamiento

El poblamiento de la base de datos se realizó utilizando sentencias INSERT INTO en PostgreSQL. Los datos insertados permiten validar el funcionamiento del modelo físico y comprobar las relaciones entre tablas mediante claves foráneas.

Además, se implementó el uso del tipo de dato JSONB en la tabla perfil para almacenar habilidades e intereses de los usuarios de forma flexible y escalable.

Los registros creados simulan un entorno real de una red social estudiantil universitaria, permitiendo posteriormente ejecutar consultas DML, vistas y procesos analíticos sobre la información almacenada.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

4.- Script de “*respaldo*” de la Bases de Datos poblada (INSERT)

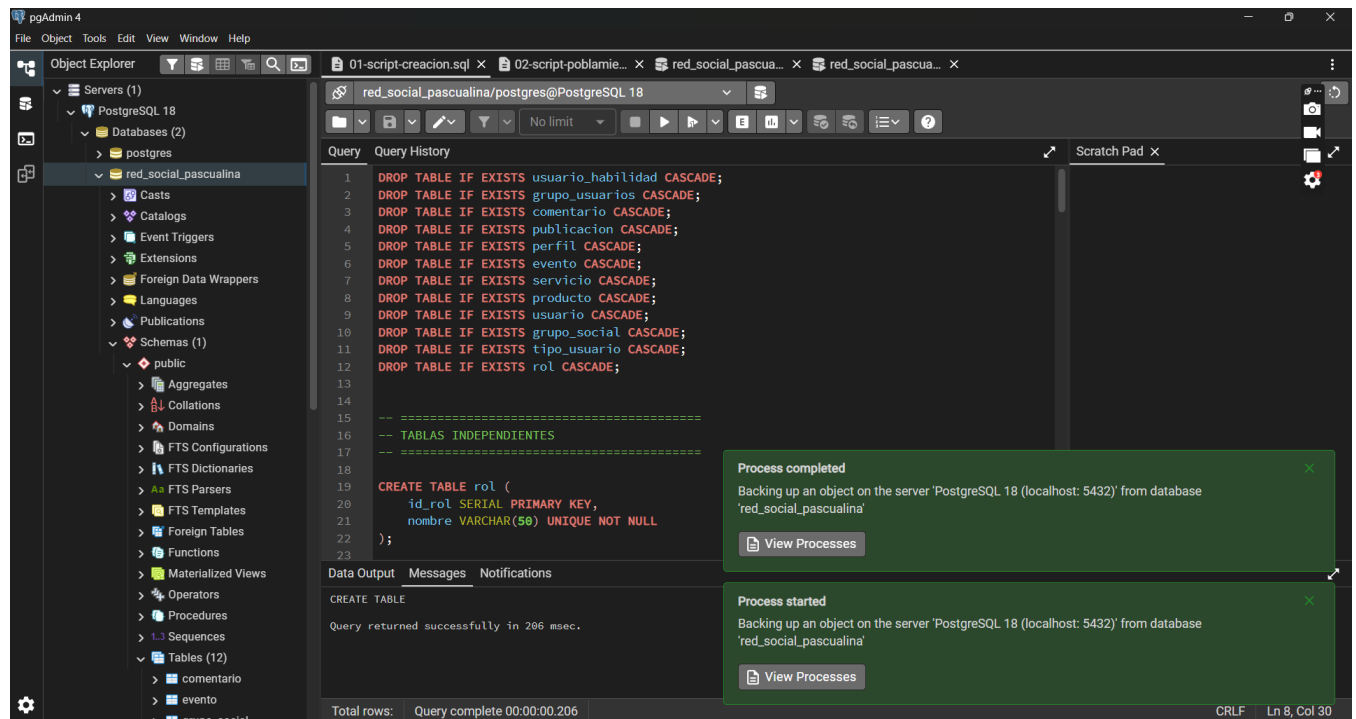
- **Script de respaldo de la base de datos (BACKUP)**

En este punto se realizó el proceso de respaldo de la base de datos *red_social_pascualina* utilizando las herramientas de administración de PostgreSQL desde pgAdmin4.

El respaldo fue generado después de ejecutar correctamente los procesos de creación y poblamiento de las tablas, garantizando la conservación de la estructura y de la información almacenada en la base de datos.

El procedimiento permitió validar el correcto funcionamiento del proceso de exportación (backup) mediante la utilidad *pg_dump*, obteniendo un archivo de respaldo que puede ser utilizado posteriormente para recuperación o restauración de la información.

La ejecución del respaldo se realizó exitosamente sin presentar errores durante el proceso.



Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

5.-Script de **actualización** de la información especificada (**UPDATE**)

Script de actualización de información (UPDATE)

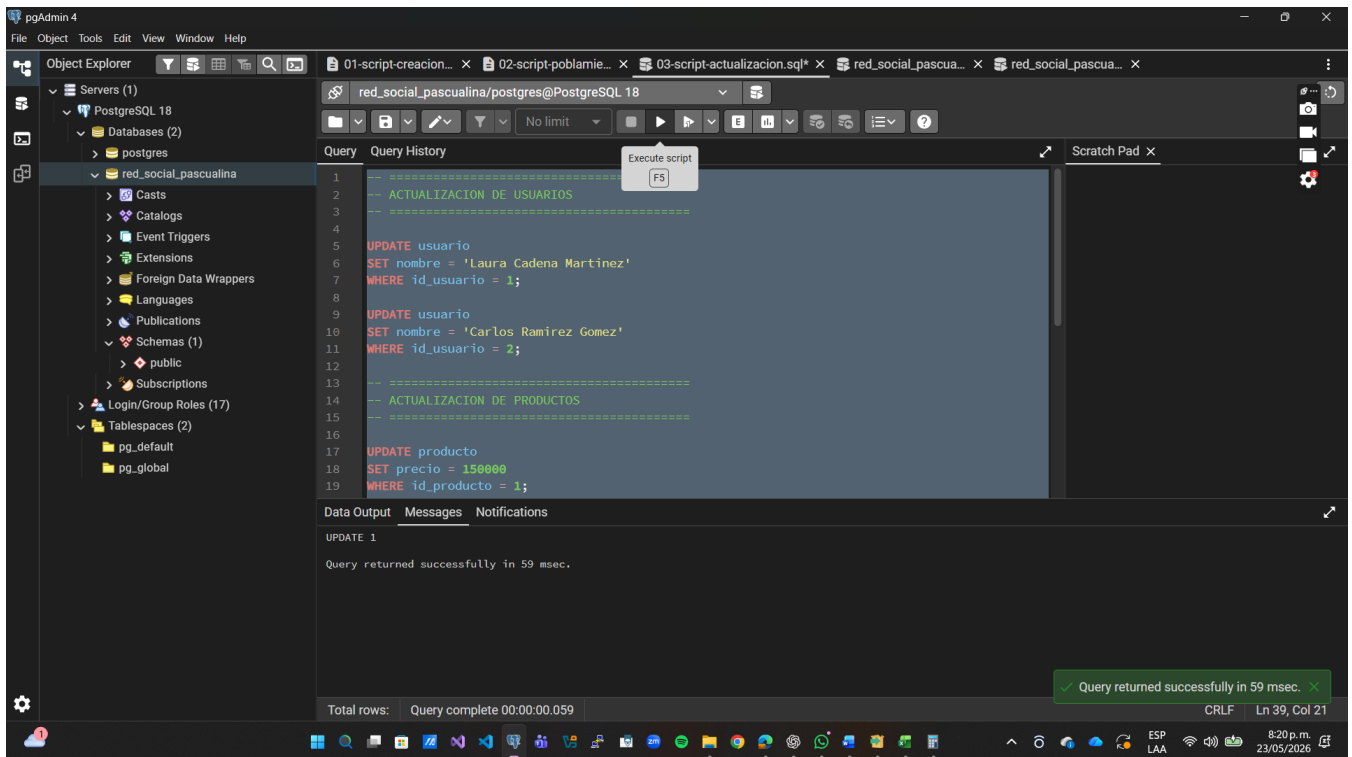
En este punto se realizaron diferentes operaciones de actualización sobre la base de datos red_social_pascualina utilizando instrucciones SQL UPDATE.

Las actualizaciones ejecutadas permitieron modificar información previamente registrada en las tablas del sistema, validando así el correcto funcionamiento de las operaciones DML en PostgreSQL.

Entre las operaciones realizadas se encuentran:

- *Actualización de información de usuarios registrados.*
- *Modificación de precios en productos publicados.*
- *Actualización de fechas correspondientes a eventos almacenados en la base de datos.*

Las sentencias fueron ejecutadas correctamente en pgAdmin4 sin presentar errores de integridad ni restricciones de claves foráneas.



Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

6.-Script de *eliminación* de la información especificada (DELETE)

Script de eliminación de información (DELETE)

En este punto se realizaron operaciones de inserción y eliminación de registros utilizando instrucciones SQL INSERT y DELETE sobre la base de datos red_social_pascualina.

Las operaciones ejecutadas permitieron registrar temporalmente información correspondiente a productos, eventos y servicios, para posteriormente eliminarlos de la base de datos debido a inconsistencias o errores detectados en los registros.

Estas operaciones permitieron validar el correcto funcionamiento de las instrucciones DML de eliminación en PostgreSQL, garantizando el manejo adecuado de la información almacenada en las tablas del sistema.

La ejecución del script se realizó correctamente en pgAdmin4 sin presentar errores de integridad referencial.

```
1  -----
2  -- INSECCION Y ELIMINACION DE PRODUCTO
3  -----
4
5  INSERT INTO producto
6  (id_usuario, nombre, precio)
7  VALUES
8  (1, 'Producto Temporal', 50000);
9
10 DELETE FROM producto
11 WHERE nombre = 'Producto Temporal';
12
13 -----
14 -- INSECCION Y ELIMINACION DE EVENTO
15 -----
16
17 INSERT INTO evento
18 (id_usuario, titulo, fecha_evento)
19 VALUES
```

Data Output Messages Notifications

DELETE 1

Query returned successfully in 54 msec.

Query complete 00:00:00.054

Query returned successfully in 54 msec.

CRLF Ln 35, Col 36

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

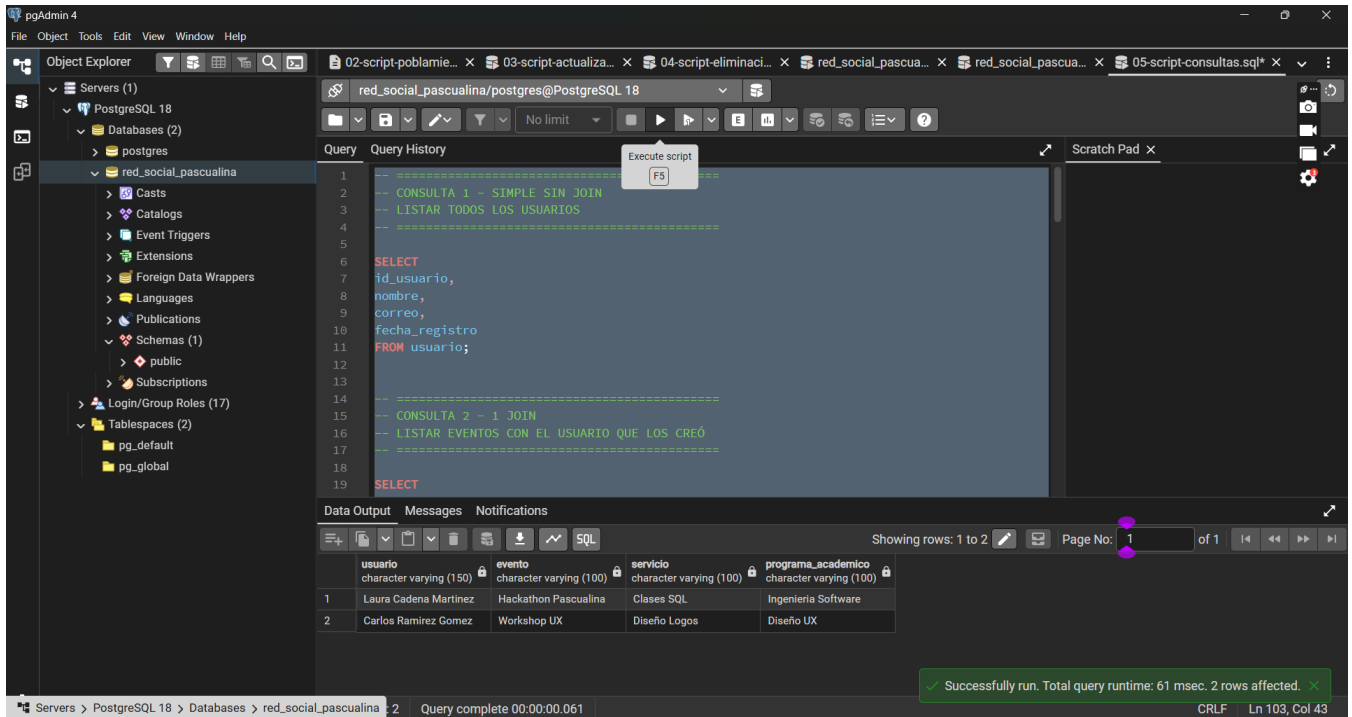
7.- Script de consultas de **listados** con la información especificada (**SELECT**).

Script de consultas de listados (SELECT)

En este punto se realizaron diferentes consultas SQL utilizando instrucciones SELECT para obtener listados de información de la base de datos “Red Social Pascualina”.

Se implementaron consultas simples y consultas con múltiples JOIN para relacionar información entre usuarios, publicaciones, comentarios, productos, servicios, eventos y grupos sociales.

Las consultas fueron desarrolladas utilizando filtros, ordenamientos y relaciones entre tablas, permitiendo validar la integridad y el funcionamiento de la base de datos.



En este punto se realizaron diferentes consultas SQL utilizando instrucciones SELECT para obtener listados de información de la base de datos “Red Social Pascualina”.

Se implementaron consultas simples y consultas con múltiples JOIN para relacionar información entre usuarios, publicaciones, comentarios, productos, servicios, eventos y grupos sociales.

Las consultas fueron desarrolladas utilizando filtros, ordenamientos y relaciones entre tablas, permitiendo validar la integridad y el funcionamiento de la base de datos.

8.- Script de consultas con **vistas** con la información especificada (**VIEW**)

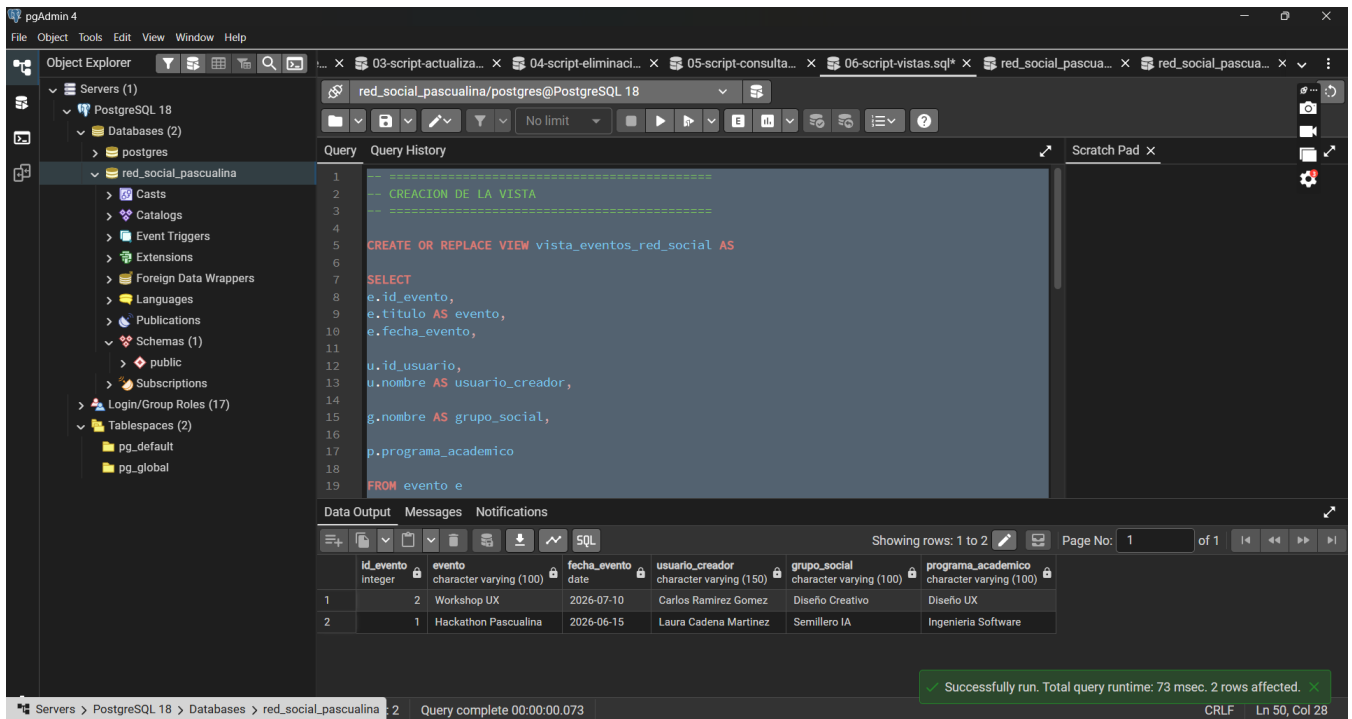
Script de consultas utilizando Vistas (VIEW)

En este punto se implementó una vista SQL utilizando la instrucción CREATE VIEW para consolidar información relacionada con eventos, usuarios y grupos sociales de la base de datos “Red Social Pascualina”.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

La vista fue creada utilizando múltiples JOIN para relacionar diferentes tablas y posteriormente se realizó una consulta sobre la vista aplicando filtros y ordenamientos.

Con este procedimiento se logró validar el uso de vistas en PostgreSQL para simplificar consultas complejas y reutilizar información estructurada de la base de datos.



En este punto se implementó una vista SQL utilizando la instrucción CREATE VIEW para consolidar información relacionada con eventos, usuarios y grupos sociales de la base de datos “Red Social Pascualina”.

La vista fue creada utilizando múltiples JOIN para relacionar diferentes tablas y posteriormente se realizó una consulta sobre la vista aplicando filtros y ordenamientos.

Con este procedimiento se logró validar el uso de vistas en PostgreSQL para simplificar consultas complejas y reutilizar información estructurada de la base de datos.

9.- Script de verificación de **propiedades ACID**

Script de verificación de propiedades ACID

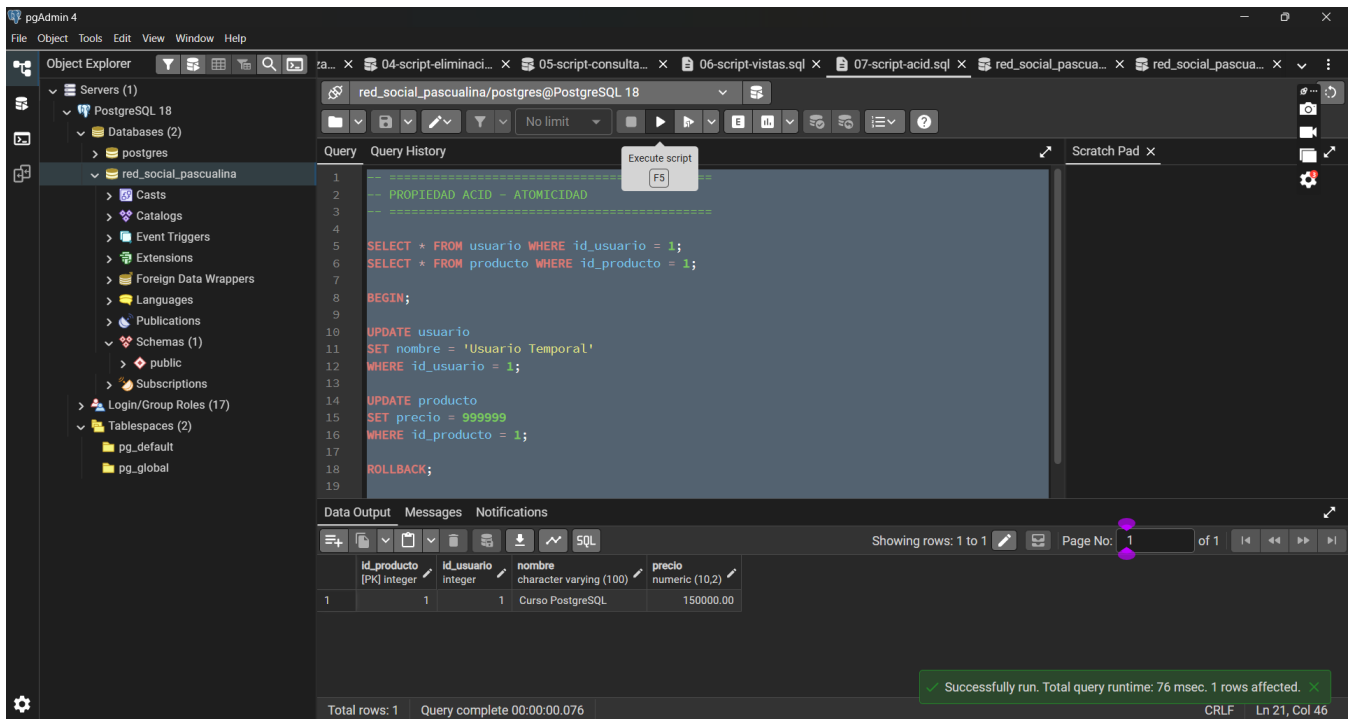
En este punto se realizaron diferentes operaciones SQL para validar las propiedades ACID de la base de datos “Red Social Pascualina”.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Se desarrollaron pruebas relacionadas con atomicidad, consistencia, aislamiento y durabilidad mediante el uso de transacciones SQL utilizando BEGIN, ROLLBACK y COMMIT.

Estas operaciones permitieron validar el comportamiento seguro y confiable de PostgreSQL durante las transacciones realizadas sobre la base de datos.

1. ATOMICIDAD (BEGIN + ROLLBACK)



The screenshot shows the pgAdmin 4 interface with the 'red_social_pascualina' database selected. The query editor contains the following SQL script:

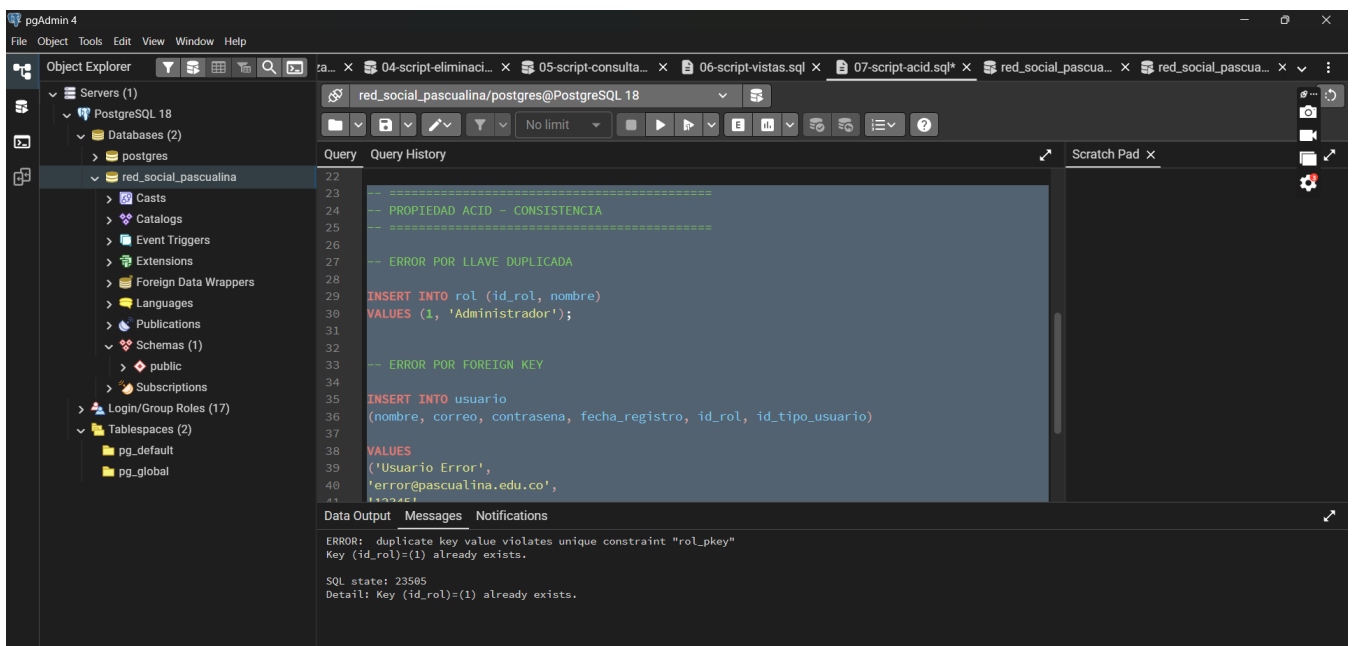
```
1  -- PROPIEDAD ACID - ATOMICIDAD
2  --
3  --
4
5  SELECT * FROM usuario WHERE id_usuario = 1;
6  SELECT * FROM producto WHERE id_producto = 1;
7
8  BEGIN;
9
10 UPDATE usuario
11 SET nombre = 'Usuario Temporal'
12 WHERE id_usuario = 1;
13
14 UPDATE producto
15 SET precio = 999999
16 WHERE id_producto = 1;
17
18 ROLLBACK;
19
```

The Data Output pane shows the results of the query:

id_producto [PK] integer	id_usuario integer	nombre character varying (100)	precio numeric (10,2)
1	1	Curso PostgreSQL	150000.00

A green message at the bottom indicates: "Successfully run. Total query runtime: 76 msec. 1 rows affected."

2. CONSISTENCIA



The screenshot shows the pgAdmin 4 interface with the 'red_social_pascualina' database selected. The query editor contains the following SQL script:

```
22
23
24 -- PROPIEDAD ACID - CONSISTENCIA
25 --
26 --
27 -- ERROR POR LLAVE DUPLICADA
28
29 INSERT INTO rol (id_rol, nombre)
30 VALUES (1, 'Administrador');
31
32 -- ERROR POR FOREIGN KEY
33
34
35 INSERT INTO usuario
36 (nombre, correo, contrasena, fecha_registro, id_rol, id_tipo_usuario)
37 VALUES
38 ('Usuario Error',
39 'error@pascualina.edu.co',
40 '123456',
41 '2023-01-01',
42 1,
43 1);
44
```

The Data Output pane shows the following error message:

```
ERROR: duplicate key value violates unique constraint "rol_pkey"
Key (id_rol)=(1) already exists.

SQL state: 23505
Detail: Key (id_rol)=(1) already exists.
```

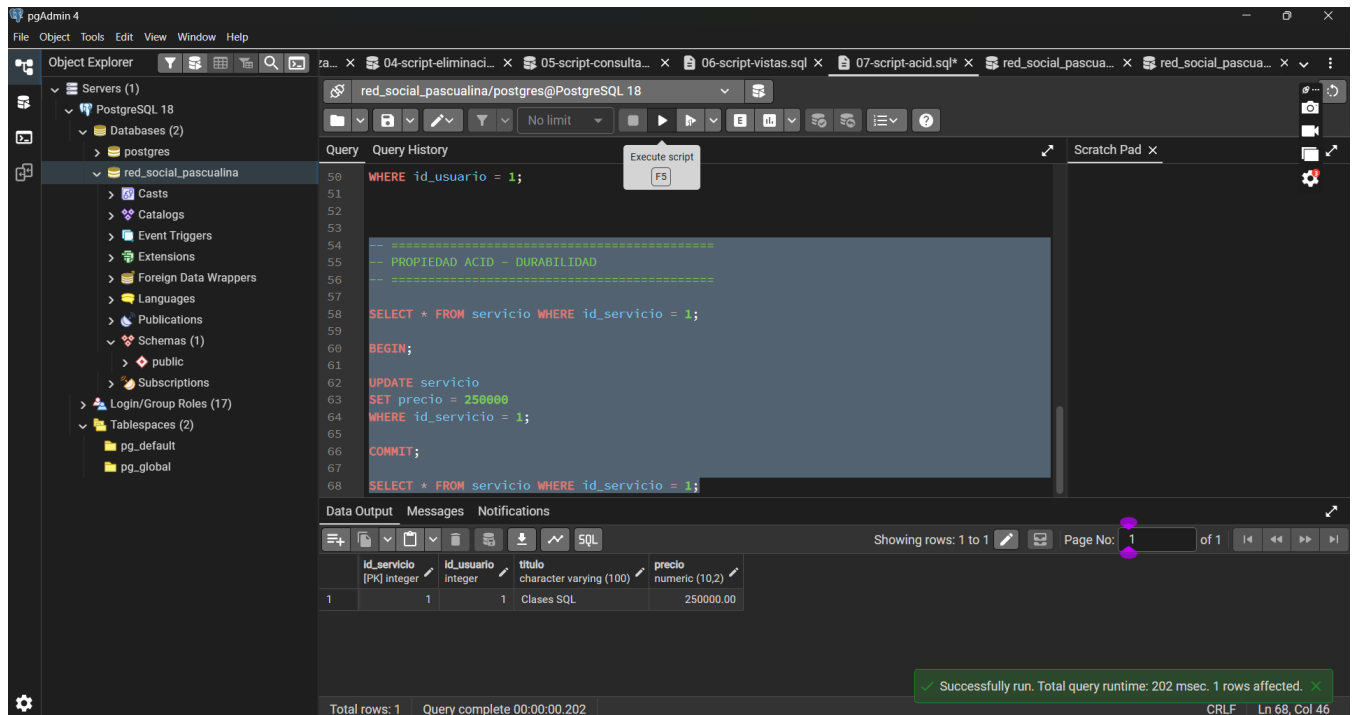
3. AISLAMIENTO

Explicación de aislamiento

La propiedad de aislamiento garantiza que múltiples transacciones ejecutadas al mismo tiempo no interfieran entre sí.

Por ejemplo, si un usuario está actualizando el precio de un producto mientras otro usuario intenta consultar ese mismo registro, PostgreSQL controla las transacciones para evitar inconsistencias y asegurar que cada operación trabaje con datos válidos y seguros.

DURABILIDAD (COMMIT)



10.- Análisis de resultados

Análisis de Resultados

Durante el desarrollo de este proyecto se realizó la construcción completa de una base de datos relacional para una red social universitaria utilizando PostgreSQL y pgAdmin4. El proceso inició con la creación del modelo físico de la base de datos, donde se definieron las tablas principales y sus relaciones mediante llaves primarias y foráneas, garantizando la integridad de la información y la correcta conexión entre los datos del sistema.

Posteriormente se ejecutaron operaciones DML como INSERT, UPDATE y DELETE, permitiendo poblar y manipular la información almacenada. En la etapa de poblamiento se registraron usuarios, roles, tipos de usuarios, productos, servicios, eventos y demás elementos relacionados con el funcionamiento de la red social. Estas operaciones permitieron comprender la importancia de mantener consistencia en los datos y respetar las restricciones definidas en las tablas.

Las consultas realizadas mediante instrucciones SELECT permitieron extraer información específica de la base de datos utilizando diferentes niveles de complejidad. Se implementaron consultas simples y consultas con múltiples JOIN, facilitando la unión de información proveniente de diferentes tablas

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

relacionadas. Gracias a esto fue posible generar reportes organizados sobre usuarios, eventos, productos y servicios, demostrando el potencial de las bases de datos relacionales para la administración eficiente de grandes volúmenes de información.

Otro aspecto importante fue la implementación de vistas (VIEW), las cuales permitieron simplificar consultas complejas y reutilizar información organizada de manera más eficiente. Las vistas ayudan a mejorar la seguridad y la facilidad de acceso a la información, además de optimizar el trabajo de consulta para futuros usuarios o desarrolladores del sistema.

Finalmente, se verificaron las propiedades ACID de la base de datos: atomicidad, consistencia, aislamiento y durabilidad. Estas pruebas demostraron que PostgreSQL garantiza transacciones seguras y confiables, evitando pérdidas o inconsistencias en la información. Las operaciones realizadas con BEGIN, ROLLBACK y COMMIT evidenciaron cómo el sistema protege los datos frente a errores o fallos durante las transacciones.

En conclusión, el desarrollo de este proyecto permitió fortalecer los conocimientos en bases de datos relacionales, consultas SQL y administración de información, comprendiendo la importancia que tienen las bases de datos en aplicaciones reales como las redes sociales y otros sistemas empresariales.

11.- Reflexiones y conclusiones individuales

Reflexiones y Conclusiones Individuales



Laura Cadena

El desarrollo de este proyecto de bases de datos permitió fortalecer mis conocimientos en PostgreSQL, consultas SQL y diseño de bases de datos relacionales. Durante la realización del trabajo aprendí la importancia de estructurar correctamente la información mediante tablas relacionadas, llaves primarias y llaves foráneas, comprendiendo cómo cada elemento cumple una función esencial dentro de un sistema de información. Aunque al inicio algunas operaciones resultaron complejas, especialmente las relacionadas con restricciones, relaciones entre tablas y propiedades ACID, poco a poco logré entender mejor el funcionamiento de las transacciones y la administración de datos en PostgreSQL.

Uno de los aspectos más importantes de este proyecto fue el aprendizaje práctico. No solamente se trabajó la teoría, sino que también se ejecutaron consultas reales utilizando operaciones DML como INSERT, UPDATE, DELETE y SELECT, además de consultas avanzadas con JOIN y creación de vistas (VIEW). Esto permitió comprender cómo se gestionan los datos en aplicaciones reales como redes sociales, sistemas empresariales y plataformas digitales. Asimismo, la validación de las propiedades ACID ayudó a entender cómo las bases de datos garantizan integridad, seguridad y confiabilidad en las transacciones. Desde el punto de vista académico, este proyecto fortaleció habilidades de análisis lógico, solución de problemas y organización de información. También permitió mejorar el manejo de herramientas como pgAdmin4 y PostgreSQL, tecnologías ampliamente utilizadas en el entorno laboral. A nivel profesional, considero que estos conocimientos serán fundamentales para mi formación como futura ingeniera de software y también aportan valor a mi experiencia en el área tecnológica y QA, ya que comprender la estructura y comportamiento de las bases de datos facilita la validación y pruebas de software.

Finalmente, este trabajo también dejó enseñanzas relacionadas con la importancia del trabajo en equipo, la responsabilidad y la organización durante el desarrollo de proyectos tecnológicos. Cada etapa requirió

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

atención al detalle, validación constante y solución de errores, demostrando que el desarrollo de bases de datos es un proceso que requiere paciencia, análisis y buenas prácticas. En conclusión, este proyecto representó una experiencia de aprendizaje muy valiosa tanto en lo académico como en lo profesional.

12.- Informe (calidad de presentación y respeto de los requerimientos)

Calidad y Presentación del Informe

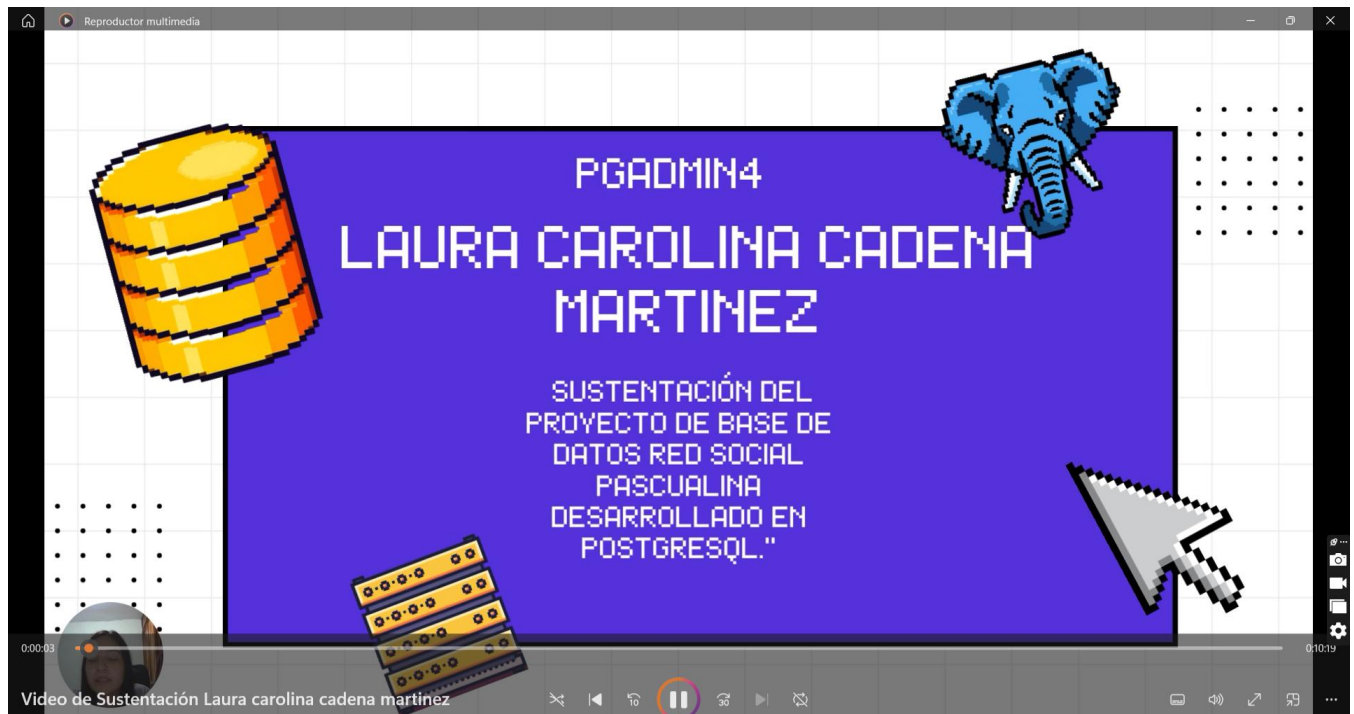
El informe fue desarrollado siguiendo una estructura organizada y coherente, incluyendo portada, introducción, desarrollo técnico, evidencias y conclusiones relacionadas con la construcción de la base de datos "Red Social Pascualina".

Durante la elaboración del documento se aplicaron criterios de presentación y organización de la información, utilizando títulos, subtítulos, evidencias visuales y fragmentos de código SQL correctamente identificados.

Asimismo, se verificó la coherencia entre el modelo conceptual, el modelo físico, los scripts SQL y las evidencias obtenidas desde PostgreSQL y pgAdmin4, garantizando consistencia en todo el proyecto desarrollado.

Finalmente, se revisaron aspectos de ortografía, redacción y formato general del documento con el propósito de entregar un informe técnico claro, ordenado y comprensible.

14.- Video de Sustentación



15.- Repositorio GIT

- [Upload files · Lau0218/bd1-20261-q100-equipo-10](#)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Rúbrica: Criterios de Evaluación de la Tarea

#	Ítems Tarea	Peso	Cal
1	Diccionario de Datos (MEJORADO y DEFINITIVO)	20	
2	Script de creación de la base de datos (CREATE)	50	
3	Script de “ poblamiento ” de las tablas especificadas (INSERT)	100	
4	Script de “ respaldo ” de las tablas pobladas (INSERT)	50	
5	Script de actualización de la información especificada (UPDATE)	10	
6	Script de eliminación de la información especificada (DELETE)	10	
7	Script de consultas de listados con la información especificada (SELECT-JOIN).	120	
8	Script de consultas con vistas con la información especificada (VIEW)	30	
9	Script de verificación de propiedades ACID	60	
10	Análisis de resultados (en equipo - 300 palabras mínimo)	30	
11	Reflexiones y Conclusiones individuales (300 palabras mínimo)	40	
12	Informe: Uso de Plantillas, Estructura, Contenido y Calidad de presentación	20	
13	Participación en Foros de Tarea	40	
14	Video de Sustentación	200	
15	Repositorio GIT	20	
	NOTA = xx/500 =	Total	800

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO A

Tabla "Rol"

Código	Rol	Descripción	Registros
1	Administrador	Tiene acceso a todos los elementos de la base de datos de manera irrestricta	10
2	Auxiliar	Tiene acceso al BackOficce de manera limitada	20
3	Miembro	Es miembro de la Red Social. Tiene permiso para hacer publicaciones, generar eventos, vender y comprar productos, vender y consumir servicios; entre otros. Pudiera tener restricciones basadas según el "Tipo de usuario".	420
4	Visitante	Puede acceder a la Red Social pero no puede interactuar de ninguna forma	50
		Total Registros de Usuarios	500

Tipo de Usuario

Código	Tipo Usuario	Descripción	Registros
1	Estudiante		300
2	Docente		70
3	Egresado		30
4	Empleado		20
5	Empresario		15
6	Ex-Empleado		5
7	Ex-Estudiante		5
8	Ex-Docente		5
9	Guess	Usuario amigo de la Red Social con algunas limitaciones	50
		Total Registros de Usuarios	500

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO B
Tipos de Servicio

Código	Tipo Servicio	Descripción
1	Asesoría académica	
2	Asesoría Laboral	
3	Curso Tecnológico	
4	Mantenimiento Moto	
5	Reparación artículo electrónico	
6	Corte de Cabello	
7	Manicure y Pedicure	
8	Reparación PC	
9	Masaje terapéutico	
10	Declaración de impuestos	
11	Asesoría Trabajo de Grado	
12	Viaje turístico	
13	Transporte	
14	Elaboración de Dulces	
15	Reparación de Calzado	
16	Confección vestimenta	
17	Organización evento	
18	Agregar nuevo	
19	Agregar nuevo	
20	Agregar nuevo	

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO C

Tipos de Productos para venta/compra

Código	Tipo Producto	Descripción
1	Libro	
2	Montocicleta	
3	Vehículo	
4	Almuerzo	
5	Desayuno	
6	Ropa	
7	Cosméticos	
8	Ticket de Concierto	
9	Bolso	
10	Zapato	
11	Postres	
12	Dulces	
13	Patineta Eléctrica	
14	Teléfono móvil	
15	Computador	
16	Articulo Deportivo	
17	Alimentos	Arroz, Pasta, entre otros
18	Agregar nuevo	
19	Agregar nuevo	
20	Agregar nuevo	

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO D
Tipos de Evento

Código	Tipo Evento	Descripción
1	Congreso	
2	Conferencia	
3	Taller	
4	Baile	
5	Fiesta	
6	Conformación de Grupo	
7	Viaje Turístico	
8	Concierto musical	
9	Reunión Semillero	
10	Feria de comida	
11	Feria de ropa	
12	Paseo en moto	
13	Feria Tecnológica	
14	Cine - Película	
15	Reunión grupo interés	
16	Entrevista laboral	
17	Matrimonio	
18	Agregar nuevo	
19	Agregar nuevo	
20	Agregar nuevo	

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO E
Propiedades ACID

Propiedad	Descripción	Ejemplo
ATOMICIDAD (Atomicity)	Una transacción: ✓ se ejecuta completamente ✗ o no se ejecuta en absoluto <i>"O todo o nada"</i>	<pre>BEGIN; UPDATE cuenta SET saldo = saldo - 100 WHERE id = 1; UPDATE cuenta SET saldo = saldo + 100 WHERE id = 2; ROLLBACK;</pre> Resultado: no pasó nada
CONSISTENCIA (Consistency)	La base de datos siempre pasa de un estado válido a otro válido. - Se respetan reglas: - claves primarias - claves foráneas - CHECK - tipos de datos <i>"Datos válidos"</i>	<pre>INSERT INTO paciente (id_paciente) VALUES (NULL);</pre> Resultado: ✗ Falla si hay restricción → mantiene consistencia
AISLAMIENTO (Isolation)	Las transacciones no se "estorban" entre sí. - Cada usuario ve su propia transacción - Evita problemas como: - lecturas sucias - lecturas fantasma <i>"Transacciones independientes"</i>	Ejemplo conceptual - Usuario A actualiza datos - Usuario B no ve cambios hasta que A haga COMMIT
DURABILIDAD (Durability)	<i>"Si se guardó, se queda guardado"</i> <i>"Persistencia"</i> <i>"COMMIT no es solo guardar... es una promesa de que nunca se perderá."</i>	<pre>BEGIN; INSERT INTO prueba_durabilidad (mensaje) VALUES ('Dato importante - no se debe perder');</pre> <pre>COMMIT;</pre> Resultado: ✓ Los datos sobreviven: - caídas del sistema - reinicios - fallos

"ACID es lo que evita que tu base de datos se convierta en caos."

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

ANEXO F

Ejemplo de inserción y consulta con datos JSON en un Tabla

Inserción de información

```
INSERT INTO apiario (nombre, ubicacion_geografica)
VALUES
(
    'Apiario La Montaña',
    '{
        "coordenadas": {
            "latitud": 6.25184,
            "longitud": -75.56359
        },
        "altitud_msnm": 1550,
        "region": {
            "pais": "Colombia",
            "departamento": "Antioquia",
            "municipio": "Medellín",
            "vereda": "Santa Elena"
        },
        "tipo_zona": "rural",
        "condiciones_ambientales": {
            "flora_dominante": ["eucalipto", "café"],
            "fuente_agua_cercana": true
        }
    }'
),
(
    'Apiario El Bosque',
    '{
        "coordenadas": {
            "latitud": 4.7110,
            "longitud": -74.0721
        },
        "altitud_msnm": 2600,
        "region": {
            "pais": "Colombia",
            "departamento": "Cundinamarca",
            "municipio": "Bogotá",
            "vereda": "Usme"
        },
        "tipo_zona": "periurbana",
        "condiciones_ambientales": {
            "flora_dominante": ["pino", "acacia"],
            "fuente_agua_cercana": false
        }
    }'
);
```

Consultas de verificación

```
SELECT * FROM apiario
WHERE ubicacion_geografica->'region'->>'departamento' = 'Antioquia';

SELECT * FROM apiario
WHERE (ubicacion_geografica->>'altitud_msnm')::int > 1500;
```